

GitHub Contributor Intelligence Service - Business Requirements

1. Legacy intent preserved

The original project required an AWS-hosted API that:

- accepts a GitHub repository and date/date-range input;
- analyzes contributors across branches;
- identifies unique domains;
- summarizes commits and contributors by domain;
- persists results in an AWS relational database;
- uses authentication and Secrets Manager;
- is version controlled, documented, reviewed and demoed with a test harness.

The 2026 specification preserves all of that intent and adds the missing rules required to make results deterministic, secure and operable.

2. Business goal

Provide a repeatable way to understand publicly observable contributor concentration and domain affiliation signals for selected repositories so users can explore potential partner/ecosystem relationships.

The output is analytical evidence, not a definitive statement of employer identity.

3. Functional requirements

ID	Requirement	Priority	Legacy mapping
GCI-FR-001	Accept repository in owner/repo format.	Must	Use Case 1/3
GCI-FR-002	Accept an ISO-8601 since and until date/time range.	Must	Use Case 3
GCI-FR-003	Support all , default , or explicit branch scope.	Must	Use Case 2
GCI-FR-004	Enumerate branches through the GitHub API when branch scope is all .	Must	Use Case 2 clarified
GCI-FR-005	Enumerate commits for each selected branch/date range with pagination.	Must	Use Case 1/2 clarified
GCI-FR-006	Deduplicate commits by immutable commit SHA across branches before aggregation.	Must	Gap filled
GCI-FR-007	Normalize contributor identity using stable GitHub identity when available; otherwise use safe fallback identifiers.	Must	Gap filled
GCI-FR-008	Derive an email domain only when the commit/API data legitimately exposes an email suitable for classification.	Must	Use Case 2 clarified
GCI-FR-009	Exclude or classify privacy/noreply domains separately; do not attempt de-anonymization.	Must	Gap filled

ID	Requirement	Priority	Legacy mapping
GCI-FR-010	Aggregate <code>total_commits</code> and <code>unique_contributors</code> per normalized domain.	Must	Example output
GCI-FR-011	Store analysis job metadata, repository, date range, run time, status and domain summary in a relational database.	Must	Scope / RDS
GCI-FR-012	Expose API endpoints to create a job, read job status and read completed summary.	Must	API service clarified
GCI-FR-013	Return structured errors with stable error codes.	Must	Gap filled
GCI-FR-014	Implement retry/backoff and rate-limit awareness for GitHub API calls.	Must	Gap filled
GCI-FR-015	Persist GitHub API/request metadata needed for troubleshooting without storing access tokens.	Should	Gap filled
GCI-FR-016	Allow repeat analyses for the same repository/date range without corrupting previous results.	Must	Gap filled
GCI-FR-017	Provide a test harness/CLI or API collection that demonstrates the service.	Must	Original scope
GCI-FR-018	Support private repositories only when the installed GitHub App has explicit access.	Should	Modern extension

4. Authentication and authorization requirements

ID	Requirement
GCI-SEC-001	Service clients must authenticate using an approved API authentication mechanism such as JWT/OIDC.
GCI-SEC-002	GitHub API access should use a GitHub App installation token for application automation rather than a long-lived shared personal access token.
GCI-SEC-003	GitHub App private key/credentials and database credentials must be stored in AWS Secrets Manager or an approved equivalent.
GCI-SEC-004	Compute must receive only the IAM permissions required for its function.
GCI-SEC-005	Tokens, authorization headers and private keys must never be written to application logs.
GCI-SEC-006	Database connections must use encryption in transit.

5. Data attribution rules

Domain attribution is inherently imperfect because GitHub users may use privacy-protected addresses, personal addresses or addresses unrelated to their current employer.

The service must therefore:

1. never claim that a domain is verified employer identity unless an explicit verified source supports that claim;
2. classify `users.noreply.github.com` and equivalent privacy addresses as non-attributable;
3. preserve a confidence/source field such as `public-email-domain`, `public-org-membership`, `unattributed`;
4. document normalization rules;
5. not scrape external personal-data sources to identify a contributor as part of the base project.

6. Non-functional requirements

ID	Area	Requirement
GCI-NFR-001	Reliability	Job state must survive API/client disconnection and be queryable later.
GCI-NFR-002	Idempotency	A client-provided idempotency key should prevent accidental duplicate job creation for retried POST requests.
GCI-NFR-003	Scalability	Design must handle pagination and avoid loading an unbounded commit history into one in-memory object.
GCI-NFR-004	Rate limits	GitHub primary/secondary rate limits must be handled without tight retry loops.
GCI-NFR-005	Performance	API job creation should return quickly; long analysis runs must execute asynchronously.
GCI-NFR-006	Observability	Emit job counts, durations, GitHub request failures/rate-limit events and worker errors.
GCI-NFR-007	Auditability	Store who/what requested a job, parameters, timestamps and terminal status when identity is available.
GCI-NFR-008	Cost	Reference AWS architecture should allow low-cost idle operation for training usage.
GCI-NFR-009	Maintainability	API schemas and database migrations must be version controlled.
GCI-NFR-010	Security	No public database endpoint is required for application operation.

7. Out of scope for base project

- determining legal employment relationship from Git activity;
- scraping LinkedIn or other third-party identity sources;
- writing/modifying GitHub repositories;
- real-time organization intelligence across all of GitHub;
- billing/payment features.

8. Legacy traceability

Legacy statement	Modern treatment
Create API Service in AWS	Preserved as GCI-FR-012 with reference AWS architecture
Fetch contributor statistics for specified repositories	Preserved and made deterministic

Legacy statement	Modern treatment
Unique domains sorted by commits/contributor across all branches	Preserved; dedupe-by-SHA added to prevent branch double count
Repository name and Date parameters	Preserved and upgraded to explicit date range
RDS table with Unique ID, Repo, Domain, Count, date range, run date	Preserved and normalized into multiple relational tables
Authentication via session token	Superseded by explicit service auth + GitHub App auth
AWS Secrets Manager	Preserved
Version control/documentation/design review/demo/test harness	Preserved